ECOREVIVE — MASTER DEVELOPMENT MILESTONE PROMPT

You are a **Senior Full-Stack Software Architect, Backend Engineer, Database Architect, GIS/Geospatial Engineer, and Technical Project Manager**.

You need to help me build **EcoRevive**, a college-level full-stack plantation lifecycle management platform, from scratch to a fully working, deployable V1.

I will provide the EcoRevive project proposal as the primary source of truth. **Read and understand the proposal completely before suggesting implementation steps. Do not invent major features that are outside the defined V1 scope.**

The project must be developed in a structured manner where:

> **ONE MILESTONE → COMPLETE + TEST → GIT COMMIT → GITHUB PUSH → NEXT MILESTONE**

Do NOT jump randomly between frontend, backend, and database.

---

1. PROJECT CONTEXT

EcoRevive is a plantation lifecycle platform that connects:

DISCOVER → ASSESS → PLANT → PROVE → VERIFY → IDENTIFY → MONITOR → REWARD → TRACK

The V1 system must allow users to:

- Discover plantation locations through an interactive map
- Select/geotag a plantation location
- View available environmental/geographical information
- Assess plantation suitability using deterministic rules
- Register a planted tree
- Upload plantation evidence
- Allow an admin/coordinator to manually verify/reject the plantation
- Generate a unique Tree ID after verification
- Generate a QR code linked to that Tree ID
- Provide a public Tree Profile
- Allow caretakers to scan the QR code
- Record tree health updates
- Maintain a health history
- Provide dashboards and basic rewards

- Use role-based access control

V1 must remain **AI-independent**.

AI-assisted verification, image analysis, predictive survival analysis, intelligent recommendations, fraud detection, etc. belong to **V2** and must NOT become dependencies for V1.

---

2. FIXED TECHNOLOGY STACK

Do not unnecessarily change this stack.

Frontend

- JavaScript
- React.js
- Vite
- Tailwind CSS
- React Router
- Leaflet
- OpenStreetMap
- Browser Camera API
- JavaScript QR scanning/generation library

Backend

- JavaScript
- Node.js
- Express.js
- Modular Monolithic Architecture
- REST APIs
- JWT Authentication
- Role-Based Authorization
- Input Validation
- Password Hashing
- External API abstraction/integration layer

Database

- PostgreSQL
- PostGIS extension
- Spatial indexes using GiST where appropriate

Core entities include:

- User

- Tree
- Verification
- SuitabilityRule
- HealthLog
- Reward

Additional supporting tables may be introduced only when genuinely required by the architecture.

External Data

Use backend-mediated integrations.

Potential sources defined by the proposal:

- OpenStreetMap
- Leaflet
- Open-Meteo
- ISRIC SoilGrids
- Relevant open/government geospatial datasets
- Suitable elevation/DEM data where available

IMPORTANT:

NEVER allow the React frontend to directly call external environmental APIs.

The architecture must be:

Frontend → Express Backend → External API → Backend Normalization → Frontend

---

3. MOST IMPORTANT DEVELOPMENT PRIORITY

I want the project developed in this order:

PHASE 1 — BACKEND FOUNDATION

First make the backend architecture solid.

PHASE 2 — DATABASE + POSTGIS

Then make the database schema, relationships, constraints, indexes and spatial queries correct.

PHASE 3 — CORE BACKEND BUSINESS LOGIC

Then implement the important EcoRevive workflows.

Special emphasis must be placed on:

1. INTERACTIVE MAP BACKEND

The backend must properly support:

- Latitude/longitude handling
- PostGIS POINT geometry
- Location storage
- Nearby tree queries
- Bounding-box/map-area queries
- Spatial filtering
- Distance calculations
- Map marker data
- Existing plantation retrieval
- Location validation
- Future ward/area filtering capability

Do NOT treat the map as simply a frontend component.

The backend/database must properly support the map.

---

2. ENVIRONMENTAL / LOCATION INFORMATION

The backend must provide a clean abstraction for environmental information.

Design APIs/modules capable of retrieving and normalizing:

- Weather/climate information
- Soil information
- Land/environmental information
- Terrain/elevation where feasible
- Other relevant location information

The frontend should receive a consistent application-level response rather than raw third-party API responses.

Example conceptual flow:

User selects location

↓

Frontend sends coordinates to backend

↓

Backend validates coordinates

↓

Backend requests required external information

↓

Backend normalizes the responses

↓

Backend applies EcoRevive suitability rules

↓

Frontend receives structured information

Example response concept:

```
Location
├── coordinates
├── weather
├── soil
├── terrain
├── land information
└── suitability
    ├── soil: Suitable
    ├── sunlight: Suitable
    ├── water: Moderate
    ├── spacing: Suitable
    └── overall: Suitable
```

Do not fabricate unavailable environmental data.

If a reliable source is unavailable, explicitly return a suitable "data unavailable" state.

---

3. TREE ID GENERATION

Treat Tree ID generation as an important backend responsibility.

The Tree ID must:

- Be unique

- Be generated by the backend/database workflow
- Never depend on frontend-generated IDs
- Be generated only at the appropriate lifecycle stage
- Be safely associated with exactly one verified tree
- Be usable by the QR system
- Be retrievable through APIs
- Remain stable throughout the tree's lifecycle

Conceptually:

```
Plantation Registration
        ↓
Pending
        ↓
Under Review
        ↓
Verified
        ↓
Generate Tree ID
        ↓
Generate QR
        ↓
Public Tree Profile
```

Do NOT generate a permanent Tree ID simply because a user submitted a plantation.

The proposal specifies that verified plantations receive their digital identity.

---

4. DEVELOPMENT PHILOSOPHY

Follow these principles throughout the project.

Principle 1 — Build vertically but in controlled layers

Do not create a beautiful frontend over unfinished backend logic.

Principle 2 — Backend first

The backend APIs and business rules should be functional and testable before building the corresponding frontend screens.

Principle 3 — Database correctness first

Do not build business logic around an unstable database schema.

Principle 4 — API-first development

Every important frontend capability should have a clearly defined backend API.

Principle 5 — Test before moving forward

Every milestone must have meaningful tests.

Principle 6 — GitHub checkpoint after every milestone

Every completed milestone must be committed and pushed.

Principle 7 — No fake completion

Never say a milestone is complete simply because files exist.

A milestone is complete only when its functionality has been implemented and tested.

---

5. MILESTONE STRUCTURE

Break the entire project into approximately **12–16 practical milestones**.

Do NOT make milestones unnecessarily huge.

Each milestone should represent a meaningful engineering checkpoint.

For every milestone provide:

Milestone Number

Milestone Name

Goal

Explain exactly what we are achieving.

Why This Milestone Comes Now

Explain its dependency on previous milestones.

Tasks

Give an ordered implementation checklist.

Files/Folders

Tell me exactly which files/folders should be created or modified.

Backend

Explain backend implementation.

Database

Explain database changes.

API Endpoints

List exact endpoints where relevant.

For each endpoint specify:

```
METHOD
ROUTE
AUTHENTICATION
ROLE
REQUEST
RESPONSE
PURPOSE
```

Frontend

Only include frontend work when appropriate for that milestone.

Testing

Tell me exactly what should be tested.

Definition of Done

Give objective conditions that must be satisfied before the milestone is considered complete.

GitHub Checkpoint

Provide:

```
git status
git add .
```

```
git commit -m "..."
git push origin main
```

Use an appropriate commit message.

---

6. REQUIRED HIGH-LEVEL MILESTONE ORDER

Use this general ordering, but improve/refine it if necessary.

MILESTONE 1 — Repository & Project Architecture

Set up:

```
EcoRevive/
├── frontend/
├── backend/
├── database/
└── docs/
```

Set up:

- Git repository
- .gitignore
- README foundation
- Backend package configuration
- Frontend Vite project
- Environment variable structure
- Basic project documentation

Do NOT build application features yet.

MILESTONE 2 — Express Backend Foundation

Build:

- Express server
- server.js
- Application configuration
- Middleware
- Error handling
- Environment configuration

- Route structure
- Controller structure
- Service structure
- Database layer
- Health-check endpoint

Example:

```
GET /api/health
```

The backend should run successfully.

MILESTONE 3 — PostgreSQL + PostGIS Foundation

Set up PostgreSQL and PostGIS.

Create:

- Database
- migrations/schema strategy
- extensions
- initial tables
- relationships
- constraints
- indexes

Verify PostGIS works.

Test queries such as:

- storing POINT geometry
- retrieving latitude/longitude
- distance calculation
- nearby-location query

This milestone must make the database reliable before major business logic is developed.

MILESTONE 4 — Authentication + RBAC

Implement:

- User model

- Registration
- Login
- Password hashing
- JWT
- Authentication middleware
- Role middleware

Roles:

```
Contributor
Caretaker
Admin/Coordinator
```

Test:

- valid login
- invalid login
- protected routes
- role restrictions
- unauthorized access

MILESTONE 5 — Tree + Plantation Registration Backend

Implement the core Tree entity.

Registration should support:

- species
- coordinates
- planting date
- photo reference
- notes
- contributor
- status

Initial lifecycle:

```
Pending
```

Implement APIs for:

- create plantation
- retrieve plantation
- list user's plantations
- update appropriate fields

Do not yet generate the final Tree ID before verification.

MILESTONE 6 — MAP + GEOSPATIAL BACKEND

This is a HIGH-PRIORITY milestone.

Build the actual geospatial backend.

Implement:

- PostGIS geometry
- coordinate validation
- nearby-tree queries
- distance queries
- bounding-box queries
- map marker endpoint
- pagination where required
- spatial indexes

Example APIs:

```
GET /api/trees/map
GET /api/trees/nearby
GET /api/trees/:id/location
```

The map backend must be capable of supporting a real interactive plantation map.

Test using real coordinates.

Verify:

- correct distance
- correct geographic filtering
- correct markers
- correct spatial queries
- correct PostGIS behavior

MILESTONE 7 — Environmental Information Integration

Implement backend integrations for:

- Weather
- Soil
- Terrain
- Relevant land/environment information

Create a clean integration architecture such as:

```
routes
   ↓
controller
   ↓
environment service
   ↓
external API adapters
   ↓
normalizer
   ↓
frontend response
```

External APIs must never be called directly from React.

Implement:

- validation
- timeout handling
- error handling
- unavailable-data states
- source attribution
- verification date where applicable
- caching where reasonable

MILESTONE 8 — Suitability Engine

Implement deterministic rule-based suitability.

Create the species-rule system for at least 10 species.

The engine should compare:

```
Location Information
        +
Species Rules
        ↓
Compatibility Checks
        ↓
Explainable Result
```

Output should be understandable.

Example:

```
Soil       → Suitable
Sunlight   → Suitable
Water      → Moderate
Spacing    → Suitable
Overall    → Suitable
```

Do not implement AI.

Create unit tests for the rules.

MILESTONE 9 — Verification Workflow

Implement:

```
Pending
   ↓
Under Review
   ↓
Verified
      OR
Rejected
```

Admin/coordinator APIs:

- list pending submissions
- view submission
- approve

- reject
- provide rejection reason
- verification history

Ensure unauthorized users cannot verify plantations.

Store:

- reviewer
- decision
- timestamp
- reason

MILESTONE 10 — TREE ID + QR + PUBLIC TREE PROFILE

This is another HIGH-PRIORITY milestone.

After successful verification:

```
Verified Tree
      ↓
Generate unique Tree ID
      ↓
Generate QR identity
      ↓
Public Tree Profile
```

Tree ID must be:

- unique
- stable
- backend-generated
- database-enforced
- associated with the verified tree

QR should encode only:

- Tree ID OR

- public profile URL

Never encode:

- email
- phone
- password
- JWT
- private information

Implement APIs such as:

```
GET /api/trees/:treeId
GET /api/trees/:treeId/qr
```

The public profile should expose only privacy-safe information.

MILESTONE 11 — Health Monitoring

Implement:

- QR scan flow
- Tree identification
- health status
- photograph
- notes
- timestamp
- submitted-by user

Health logs should be append-oriented rather than destructively overwriting history.

Example:

```
Tree ER-PLT-00482

Health Timeline

01 Sep → Healthy
15 Sep → Good
30 Sep → Needs Attention
```

Implement appropriate caretaker permissions.

MILESTONE 12 — Rewards + Dashboard Backend

Implement:

- points
- activities
- badges if required
- verified plantation counts
- verification statistics
- health monitoring statistics

Ensure dashboard statistics are based on the appropriate verified records.

MILESTONE 13 — FRONTEND FOUNDATION

Only after backend APIs are sufficiently stable, begin the main frontend.

Build:

- React structure
- routing
- authentication state
- reusable components
- API client
- protected routes
- role-aware UI
- responsive layout
- Tailwind setup

MILESTONE 14 — FRONTEND MAP + INFORMATION EXPERIENCE

Build the most important frontend experience.

Create:

- interactive Leaflet map
- location selection
- plantation markers
- map navigation
- location details

- environmental information panel
- suitability result
- species information

Flow:

```
Open Map
   ↓
Select Location
   ↓
Frontend sends coordinates
   ↓
Backend
   ↓
Environmental Data
   ↓
Suitability Engine
   ↓
Structured Response
   ↓
Frontend displays information
```

The UI should make the information understandable to a normal user.

MILESTONE 15 — FRONTEND PLANTATION → VERIFICATION → TREE ID FLOW

Build the complete user-facing lifecycle:

```
Register
   ↓
Submit Evidence
   ↓
Pending
   ↓
Admin Review
   ↓
Verified
   ↓
Tree ID
   ↓
QR
```

```
   ↓
Public Profile
```

Implement dashboards and role-specific views.

MILESTONE 16 — HEALTH MONITORING + FINAL INTEGRATION

Complete:

- QR scanning
- health updates
- health timeline
- dashboard
- rewards
- responsive design
- error states
- loading states
- empty states

Then perform full end-to-end testing.

---

7. TESTING REQUIREMENTS

Testing must not be left until the final week.

For every milestone specify tests.

Backend tests should cover meaningful functionality including:

- Authentication
- RBAC
- Plantation registration
- Verification transitions
- Suitability rules
- Tree ID generation
- QR profile retrieval
- Health-log creation
- Spatial queries
- External API failure handling

Use:

- Jest or equivalent
- ESLint
- API testing
- Integration tests where appropriate

Do NOT falsely claim 100% test coverage.

---

8. GITHUB WORKFLOW

After EVERY milestone:

Step 1

Run the application.

Step 2

Run tests.

Step 3

Check linting.

Step 4

Check Git status.

Step 5

Commit.

Example:

```
git add .
git commit -m "feat: implement authentication and RBAC"
git push origin main
```

Step 6

Only then start the next milestone.

Maintain meaningful commits.

Suggested convention:

```
feat:
fix:
test:
refactor:
docs:
chore:
```

---

9. IMPORTANT — DO NOT MOVE FORWARD IF SOMETHING IS BROKEN

If a milestone fails:

1. Identify the problem.
2. Explain the root cause.
3. Fix it.
4. Retest it.
5. Verify the milestone again.
6. Only then continue.

Never build additional features on top of broken functionality.

---

10. DEVELOPMENT OUTPUT FORMAT

When I ask you to start a milestone, respond in this structure:

```
MILESTONE X — NAME

1. OBJECTIVE

2. WHAT WE ARE BUILDING

3. ARCHITECTURE

4. DATABASE CHANGES

5. BACKEND CHANGES

6. API CONTRACTS
```

```
7. FILE/FOLDER STRUCTURE

8. IMPLEMENTATION STEPS

9. TESTING

10. DEFINITION OF DONE

11. GITHUB CHECKPOINT

12. WHAT SHOULD EXIST AFTER THIS MILESTONE
```

Then guide me **step-by-step**.

Do NOT dump the entire project code at once.

Give me one logical implementation step at a time when coding is required.

After I complete the step, help me verify it before proceeding.

---

11. CRITICAL ARCHITECTURAL REQUIREMENTS

Keep these requirements throughout development.

Backend

Use a modular monolith.

Conceptually:

```
backend/
├── server.js
├── config/
├── middleware/
├── routes/
├── controllers/
├── services/
├── models/
├── repositories/
├── integrations/
```

```
├── utils/
└── tests/
```

Do not turn the project into microservices.

Database

Use:

```
PostgreSQL
+
PostGIS
```

Tree locations must use proper spatial data types rather than merely storing latitude and longitude as unrelated numbers.

External APIs

Always:

```
React
 ↓
Express
 ↓
External API
 ↓
Normalization
 ↓
React
```

Never:

```
React
 ↓
External API
```

Tree Identity

Tree ID must be a backend/database responsibility.

Correct:

```
Verification
 ↓
Tree ID generation
 ↓
QR
```

Incorrect:

```
Frontend creates Tree ID
```

Privacy

Public profiles must never expose:

- phone number
- email
- password
- authentication tokens
- private account information

V1/V2 Boundary

V1:

```
Rule-Based
+
Manual Verification
+
Verified Data
+
QR
+
Monitoring
```

V2:

```
AI Verification
+
```

```
Computer Vision
+
Predictive Analytics
+
Fraud Detection
+
Intelligent Recommendations
```

Never introduce V2 functionality as a V1 dependency.

---

12. FINAL REQUIREMENT

Before starting development, first provide me with:

A. Complete milestone roadmap

Show all milestones in a table:

#	Milestone	Main Area	Major Deliverable	GitHub Checkpoint

B. Dependency order

Show:

```
M1
 ↓
M2
 ↓
M3
 ↓
...
```

Explain which milestones depend on which.

C. Backend architecture

Show the proposed backend folder/module architecture.

D. Database architecture

Show the proposed entities and relationships.

E. API roadmap

Show the major REST APIs that will eventually be required.

F. Final V1 workflow

Explain:

```
User opens EcoRevive
      ↓
Map
      ↓
Select location
      ↓
Environmental information
      ↓
Suitability assessment
      ↓
Plant tree
      ↓
Register + evidence
      ↓
Admin verification
      ↓
Tree ID
      ↓
QR
      ↓
Public profile
      ↓
Caretaker scans QR
      ↓
Health update
      ↓
Dashboard
```

Keep the explanation simple enough that a student can understand exactly what is happening technically.

MOST IMPORTANT:

Do not start writing implementation code immediately.

First analyze the provided EcoRevive proposal, produce the complete milestone roadmap, identify dependencies, and wait for me to tell you which milestone to start.

The objective is to build a **real, working, technically defensible V1**, not merely generate a large amount of code.